

Implementación Local de Modelos de Lenguaje de Gran Escala (LLM)

I. Introducción

En la actualidad, el despliegue de **Modelos de Lenguaje de Gran Escala (LLM)** en entornos locales se ha convertido en una alternativa fundamental frente a las soluciones basadas en la nube. Esta práctica permite ejecutar modelos avanzados sin depender de una conexión a internet constante, garantizando la privacidad de los datos y reduciendo los costos operativos de API externas.

II. Herramientas

- **El modelo Llama 3.2**

Llama 3.2 es desarrollada por Meta. Se distingue por su eficiencia en el uso de parámetros, permitiendo que dispositivos con recursos de hardware estándar puedan procesar tareas complejas de lenguaje natural con una latencia mínima.

- **PowerShell y CLI**

El uso de la interfaz de línea de comandos (CLI) a través de Windows

III. Metodología

El proceso de implementación se llevó a cabo mediante el uso de la interfaz de línea de comandos de **Windows PowerShell**, utilizando el gestor de modelos **Ollama**.

A. Instalación del Entorno de Ejecución

Como se observa en la parte superior de la imagen [1], la instalación se inició mediante el comando: `irm https://ollama.com/install.ps1 | iex` Este comando realiza una petición web para descargar el script de instalación y ejecutarlo directamente en el sistema, configurando las variables de entorno necesarias para el funcionamiento del servidor local de inferencia.

B. Gestión y Descarga del Modelo Llama 3.2

Tras verificar la instalación con el comando de ayuda (`ollama -h`), se procedió a la ejecución del modelo:

1. **Comando de ejecución:** Se utilizó `ollama run llama3.2`.

C. Prueba de Inferencia y Validación

Una vez completada la escritura del manifiesto el modelo queda residente en la memoria local.

- **Interacción inicial:** Se realizó un saludo ("hola"), donde el modelo respondió confirmando la correcta configuración del lenguaje.
- **Prueba de lógica y programación:** Se solicitó un algoritmo de ordenamiento

específico (Selection Sort), excluyendo explícitamente el método de burbuja. Como muestra la captura, Llama 3.2 fue capaz de procesar la restricción y generar código funcional **en Python**, demostrando capacidades de razonamiento lógico y asistencia en programación de manera local.

IV. Resultados Obtenidos

La implementación fue exitosa. La captura de pantalla evidencia que el sistema anfitrión tiene la capacidad de:

1. **Soportar la carga de trabajo de un modelo de 3B o similar.**
2. **Gestionar peticiones en tiempo real** sin necesidad de latencia de red externa.
3. **Producir sintaxis de código correcta** Cumpliendo con los parámetros de la solicitud del usuario.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\Users\nanci> irm https://ollama.com/install.ps1 | iex
>>> Downloading Ollama for Windows...
##### 100.0%
>>> Installing Ollama...
>>> Install complete. Run 'ollama' from the command line.
PS C:\Users\nanci> ollama -h
Large language model runner

Usage:
  ollama [flags]
  ollama [command]

Available Commands:
  serve      Start Ollama
  create      Create a model
  show        Show information for a model
  run         Run a model
  stop        Stop a running model
  pull        Pull a model from a registry
  push        Push a model to a registry
  signin      Sign in to ollama.com
  signout     Sign out from ollama.com
  list        List models
  ps          List running models
  cp          Copy a model
  rm          Remove a model
  launch      Launch the Ollama menu or an integration
  help        Help about any command
```

```
Flags:
  -h, --help            help for ollama
  --nowordwrap          Don't wrap words to the next line automatically
  --verbose             Show timings for response
  -v, --version         Show version information

Use "ollama [command] --help" for more information about a command.
PS C:\Users\nanci> ollama list
NAME      ID          SIZE    MODIFIED
PS C:\Users\nanci> ollama run llama3.2
pulling manifest
pulling dde5aa3fc5ff: 100%
pulling 966de95ca8a6: 100%
pulling fcc5a6bec9da: 100%
pulling a70ff7e570d9: 100%
pulling 56bb8bd477a5: 100%
pulling 34bb5ab01051: 100%
verifying sha256 digest
writing manifest
success
>>>
>>> hola
Hola! ¿En qué puedo ayudarte hoy?

>>> me gustaria que me dieras un codigo de ordenamiento, menos el de burbuja.
Claro, te puedo proporcionar algunos ejemplos de algoritmos de ordenamiento, excluyendo el de burbuja. Aquí te dejo algunos opciones:

**1. Algoritmo de selección**

```python
def ordenamiento_seleccion(arr):
 n = len(arr)
 for i in range(n):
 min_idx = i
 for j in range(i+1, n):
 if arr[j] < arr[min_idx]:
 min_idx = j
 arr[i], arr[min_idx] = arr[min_idx], arr[i]
 return arr
```